

Full Stack Task: Password Strength Meter (React + Node.js)

Objective

You are given a working HTML + CSS + JavaScript file (Demo.html). First open in the browser and see how it works.

Your goal is to:

1. Convert it into **React (single component)**
2. Create a **simple Node.js backend (single file)**
3. Connect both so data is **saved and loaded automatically**

Do not change the design. Focus on logic and integration.

Note: A small “Syntax Appendix” is provided at the very end of this document (fetch GET/POST + Node/Express file read/write) in case you forget exact syntax.

Part 1: React (Single Component)

What You Need To Do

- Copy the given HTML into JSX
 - Use the same CSS (no UI changes)
 - Implement all logic using React (useState, useEffect)
 - Keep everything inside **one component only**
-

Functional Requirements

Password Input

- Default type: password

Show / Hide Toggle

- Button shows:
 - Show (when hidden)
 - Hide (when visible)
- Clicking toggles input type

Password Strength Rules

- Less than 6 → Weak

- 6 to 9 → Medium
- 10 or more AND contains @ or # → Strong

Real-Time Update

- Strength updates on every keystroke
- Must NOT depend on button click

Strength UI

- Show:
 - Strength label
 - Colored badge
 - Progress bar

Reset Behavior

- If input is empty:
 - Label becomes -
 - Progress becomes 0%

Theme Selector

- 5 themes available
 - First theme selected by default
 - Clicking a theme:
 - Changes background
 - Highlights active theme
-

Part 2: Node.js (Single File)

What You Need To Do

- Create one file: `server.js`
- Use `fs` module to store data in a JSON file

Data to Store

- password
- selected theme

Example:

```
{  
  "password": "abc@12345",  
  "theme": "theme-2"
```

```
}
```

API Requirements

GET /data

- Returns saved password and theme

POST /data

- Accepts:

```
{  
  "password": "...",  
  "theme": "..."  
}
```

- Saves data into file
-

Part 3: React + Backend Integration

On Page Load

- Fetch data from backend (GET /data)
- Fill:
 - Password input
 - Selected theme
 - Strength UI

On User Change

Whenever user: - Types password OR - Changes theme

→ Send updated data to backend (POST /data)

Important Rules

- Use only ONE React component
 - Do NOT use DOM methods (no querySelector)
 - Do NOT change CSS
 - Keep code clean and simple
-

Submission Instructions (Very Important)

Students must submit ONLY the following 2 files:

- `App.jsx` (a single React component file containing the complete UI + logic)
- `server.js` (Node.js backend in a single file)

Not allowed:

- No zip/rar files
 - No folders (no React project folder)
 - No extra files (no `package.json`, no screenshots, no multiple components)
-

Checklist (Self Verification)

React

- ☐ HTML correctly converted to JSX
 - ☐ CSS reused without changes
 - ☐ Password input works
 - ☐ Show/Hide toggle works correctly
 - ☐ Strength updates on typing
 - ☐ Correct strength logic implemented
 - ☐ Strength resets when input is empty
 - ☐ Badge color updates correctly
 - ☐ Progress bar updates correctly
 - ☐ Theme selection works
 - ☐ Default theme applied on load
 - ☐ Active theme is highlighted
 - ☐ No direct DOM manipulation used
 - ☐ Everything inside ONE component
-

Backend

- ☐ Single Node.js file created
- ☐ JSON file used for storage
- ☐ GET /data endpoint works
- ☐ POST /data endpoint works

- ☐ Data is saved correctly in file
-

Integration

- ☐ React fetches data on load
 - ☐ UI pre-fills with saved data
 - ☐ Changes are sent to backend
 - ☐ Theme persists after refresh
 - ☐ Password persists after refresh
-

Syntax Appendix (Minimal Reference)

This section is only for syntax reference. Do not copy/paste blindly; adapt it to the task.

1) Fetch API (GET + POST JSON)

Example scenario (different from the task): saving a user's preferred language.

```
// POST example
async function savePrefs(language) {
  const res = await fetch("http://localhost:3001/foo", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ language }),
  });
  if (!res.ok) throw new Error("POST failed");
  return await res.json(); // e.g. { ok: true }
}
```

2) Node.js: Read/Write JSON File (fs)

Example scenario (different from the task): persisting a counter value.

```
const fs = require("fs");
const path = require("path");

const filePath = path.join(__dirname, "counter.json");

function readCounter() {
  try {
    const raw = fs.readFileSync(filePath, "utf8");
    return JSON.parse(raw); // { count: 3 }
  } catch (err) {
    return { count: 0 }; // default if file missing/invalid
  }
}

function writeCounter(next) {
  fs.writeFileSync(filePath, JSON.stringify(next, null, 2), "utf8");
}
```